Hardware/Software Interface" and is available on Coursera.org [*https://www.coursera.org/course/hwswinterface].*

Another great resource of free full courses on Assembly at available at opensecuritytraining.info. They have courses on x86, x86-64, and ARM at an introductory, intermediate and advanced level.

Instead of jumping straight into Core i7, you should start by looking into programming the 8-bit Intel 8085 processor. It might be nearly 40 years old, but it has a design that you can conceivably hold in your head all at once. From there you can build up to 16, 32, and finally today's 64bit processors.

Once you know a bit of Assembly language, you can start playing around with your own code. For this you will need an Assembler, a piece of software that converts assembly code to machine code. A popular assembler is the Netwise Assembler (NASM) that can run on Windows, OSX or Linux.

The popular Gnu Compiler Collection (GCC) includes support for assembly language, and even even compile higher-level code to Assembly, so you can see what your code looks like in assembly. Visual Studio too ships with an assembler called MASM (Microsoft Assembler).

Another way to interact with Assembly code is by disassembling existing code! Decompiling code into its source code is hard to impossible, but getting assembly code for compiled software is easy. On POSIX platforms like OSX / Linux / Unices you can simply type objdump -d /path/to/compiled/binary to get the Assembly representation of compiled code.

Finally, if you want to play with running code and see as it executes in Assembly, you have gdb possibly the most powerful debugging tool in existence. To end we'd like to mention that Assembly is a good place to start at a conceptual level, it should be the last place you come to to actually get your hands dirty with code. Come back to this chapter again once you get familiar with higher level languages that follow.